

Competitive Programming

Section 1: Complexity, I/O, and constraints

Competitive Programming Course

2026-07-04

Table of contents

1	Why constraints matter	3
2	The first reading of a statement	4
2.1	Two different kinds of constraints	4
3	A practical operation budget	5
3.1	A more concrete budget calculation	6
3.2	Big-O is not a stopwatch	6
4	Example: summing all numbers	7
4.1	Trace on a small input	7
5	Integer size	8
5.1	C++ integer checklist	8
6	Fast input	9
6.1	Output buffering	9
6.2	Parsing patterns	9
7	Example: simulation until a threshold	11
7.1	Why not derive a formula immediately?	11
8	Example: recognizing impossible complexity	12
8.1	Worked decisions	12
9	Proof habits start now	13
10	Using the local judge	14
11	Common mistakes	15
12	Before you submit	16
13	Exercises	17

1 Why constraints matter

In competitive programming, the first algorithm you think of is often not the algorithm you are allowed to submit. The statement tells you the input size; the time limit tells you roughly how many operations you can afford. Your job is to connect those two facts before writing code.

This section trains three habits:

1. estimate the largest input size;
2. pick a plausible complexity class;
3. write input/output code that does not dominate the solution.

The lesson is deliberately about small problems. That is not because small problems are the destination. It is because red-level problem solving still starts with the same first question:

What is the largest computation this statement permits?

If that question is answered incorrectly, the rest of the solution is built on bad ground. A clever proof does not save an algorithm that needs 10^{14} operations.

2 The first reading of a statement

When you open a contest problem, do not begin by coding. Spend a short, fixed amount of time extracting the operational facts.

```
statement
|
+-- input size: n, m, q, sum of n, number of tests
|
+-- value size: 10^9, 10^18, modulo, negative values
|
+-- output shape: one answer, one per test, object construction
|
+-- hidden structure: sorted? graph? tree? sequence? grid?
```

For this section, the hidden structure is simple. We are training the first two branches: input size and value size.

2.1 Two different kinds of constraints

A common beginner mistake is to read all numbers in the constraints as if they mean the same thing. They do not.

Constraint kind	Example	What it controls
Count	<code>n <= 200000</code>	number of loop iterations or elements
Value	<code>ai <= 10^9</code>	integer size and overflow risk
Sum bound	<code>sum n <= 200000</code>	total work over all tests
Query count	<code>q <= 200000</code>	number of times a data structure is used
Time limit	<code>2 seconds</code>	rough operation budget

The count constraints usually decide the algorithm. The value constraints usually decide arithmetic and sometimes suggest number theory or binary search. The sum bound prevents you from multiplying every maximum together.

3 A practical operation budget

Operation counts are not exact. A C++ loop doing simple integer arithmetic may run hundreds of millions of iterations per second on a contest machine. Python is often an order of magnitude slower. A graph algorithm with cache misses or a heavy constant factor may be slower than the formula suggests.

The following table is a contest heuristic, not a law. Use it to reject ideas, not to prove that an implementation will pass.

Maximum main size	Candidate target	Typical examples
$n \leq 20$	$O(2^n)$, $O(n \cdot 2^n)$	subsets, bitmask DP
$n \leq 80$	$O(n^4)$ sometimes	dense DP, Floyd-like variants
$n \leq 500$	$O(n^3)$ may pass	Floyd-Warshall, interval DP
$n \leq 2,000$	$O(n^2)$	pairs, grid DP, simple DP
$n \leq 200,000$	$O(n \log n)$ or $O(n)$	sorting, heaps, Fenwick trees
$n \leq 1,000,000$	$O(n)$	sieves, prefix scans
$n \leq 10^9$	$O(\log n)$ or formula	binary search, math
$n \leq 10^{18}$	$O(\log n)$, digit count, math	Euclid, exponentiation

The important action is not memorizing the table. The important action is rejecting impossible ideas early.

One useful mental diagram is:

```
constraints
  |
  v
largest possible input
  |
  v
operation budget estimate
  |
  v
allowed complexity class
  |
  v
implementation choice
```

When a contest problem feels mysterious, return to this chain. Many wrong solutions fail at the second or third box.

3.1 A more concrete budget calculation

Suppose a statement has:

```
t <= 10000
n <= 200000
sum of n over all tests <= 200000
```

There are two tempting but wrong reactions:

- $t * n = 2,000,000,000$, so even reading input is too large.
- $n = 200000$, so every test can do an $O(n \log n)$ sort independently.

The correct reading is more careful. Across the entire input, there are only 200,000 array elements. A linear scan over every element is fine. Sorting every test case separately costs

$$\sum_i n_i \log n_i,$$

which is at most roughly

$$\left(\sum_i n_i \right) \log \left(\sum_i n_i \right)$$

for this kind of estimate, so sorting each case can also be plausible.

However, an $O(n_i^2)$ loop inside each case is dangerous. If one test has $n = 200000$, that single case already needs $4 * 10^{10}$ pair checks.

3.2 Big-O is not a stopwatch

Big-O hides constants and lower-order terms. These two snippets are both linear, but they do not cost the same:

```
for x in array:
    sum += x
```

```
for x in array:
    insert x into a hash table
    allocate a small object
    update several nested containers
```

At this level, the rule is:

- use Big-O to choose the family of algorithm;
- use implementation judgment to make the chosen family fast enough.

4 Example: summing all numbers

Suppose the input contains t test cases, and the total number of integers over all test cases is at most 200,000. The required answer is the sum in each case.

A correct algorithm is:

```
for each test case:
    read n
    read n values
    print their sum
```

The total work is linear in the total amount of input:

$$O\left(\sum n_i\right).$$

This is the right way to reason about multiple test cases. If each case has $n \leq 200000$ and $t \leq 200000$, but the statement also says the sum of all n is at most 200000, then the real bound for reading the arrays is 200000, not 40 billion.

4.1 Trace on a small input

Input:

```
3
5
1 2 3 4 5
1
-10
4
1000000000 1000000000 1000000000 1000000000
```

The computation is:

case	values	running idea	answer
1	1 2 3 4 5	$0+1+2+3+4+5$	15
2	-10	$0-10$	-10
3	four times 10^9	needs 64-bit	4000000000

The third answer is the trap. It does not fit in signed 32-bit `int`.

5 Integer size

Contest integers have two separate constraints:

- the value of each input number;
- the value of an intermediate or final answer.

If $n = 200000$ and each value can be as large as 10^9 , the sum may be

$$2 \cdot 10^5 \cdot 10^9 = 2 \cdot 10^{14}.$$

That does not fit in a 32-bit signed integer. In C++, use `long long`. Python integers grow automatically, but that does not remove the need to know whether the answer is supposed to be printed exactly or modulo some number.

5.1 C++ integer checklist

Use `int` for indices and small counts when the maximum is around $2 \cdot 10^5$. Use `long long` for values, sums, products, distances, and answers that may reach 10^{18} .

For multiplication, casting after the multiplication is too late:

```
int a = 1000000000;
int b = 1000000000;
long long wrong = a * b;           // overflow already happened
long long right = 1LL * a * b;    // multiplication is 64-bit
```

For products that may exceed $9 \cdot 10^{18}$, use `__int128` in C++ for the intermediate comparison. Problem C uses this idea because n^3 may be 10^{27} , even though the budget is at most 10^{18} .

6 Fast input

For C++:

```
ios::sync_with_stdio(false);
cin.tie(nullptr);
```

For Python:

```
import sys
data = sys.stdin.buffer.read().split()
```

The Python pattern reads all tokens at once. It is excellent for ordinary integer-heavy contest input. For interactive problems it is wrong, but this course does not start with interactive tasks.

6.1 Output buffering

Printing one line at a time is usually fine in C++ when streams are untied. In Python, build a list of strings and join it:

```
out = []
for answer in answers:
    out.append(str(answer))
sys.stdout.write("\n".join(out))
```

This avoids paying a separate high-level print cost for every answer.

6.2 Parsing patterns

For fixed-size records:

```
data = list(map(int, sys.stdin.buffer.read().split()))
idx = 0
t = data[idx]
idx += 1
for _ in range(t):
    x = data[idx]
    y = data[idx + 1]
    idx += 2
```

For variable-length arrays:

```
data = list(map(int, sys.stdin.buffer.read().split()))
idx = 0
t = data[idx]
idx += 1
for _ in range(t):
    n = data[idx]
    idx += 1
    values = data[idx:idx + n]
    idx += n
```

Slicing copies the list segment. That is fine for many easy tasks, but for large inputs you can sum directly while advancing the index.

7 Example: simulation until a threshold

Consider this process:

```
x starts below or equal to y.  
One operation changes x to 2x + 1.  
How many operations are needed until x >= y?
```

If $x = 3$ and $y = 30$, the values are:

```
3 -> 7 -> 15 -> 31
```

The answer is 3.

The loop is not proportional to y ; the value roughly doubles. The number of iterations is logarithmic:

$$O(\log y).$$

This distinction is one of the first contest filters. A loop that increments by 1 may be impossible for $y = 10^{18}$; a loop that doubles may be trivial.

The growth pattern is:

operation count:	0	1	2	3	4
value:	x	$2x+1$	$4x+3$	$8x+7$	$16x+15$

Each row doubles the previous scale, so the number of rows is small.

7.1 Why not derive a formula immediately?

After k operations, the value is:

$$2^k x + (2^k - 1).$$

You could solve an inequality involving logarithms. But for $y \leq 10^{18}$, the loop runs at most about 60 times from positive x . The loop is simpler, less error-prone, and easily fast enough.

This is an important contest habit: do not replace a tiny logarithmic loop with a fragile formula unless the formula gives a real advantage.

8 Example: recognizing impossible complexity

You are given a proposed input size n and an operation budget B . Which of these algorithms fit?

```
O(n)
O(n log n)
O(n^2)
O(n^3)
```

For contest estimates, we can use `ceil(log2(max(1, n)))` for the logarithm and compare against B .

This is not how formal complexity theory defines Big-O. It is a practical model that helps you choose an implementation under pressure.

8.1 Worked decisions

If $n = 100000$ and $B = 2000000$, then:

```
n          = 100000
log2(n)    = about 17
n log n    = about 1700000
n^2        = 10000000000
```

The slowest fitting class is $n \log n$. A quadratic algorithm is not close.

If $n = 2000$ and $B = 4000000$, then:

```
n^2 = 4000000
n^3 = 8000000000
```

Quadratic fits exactly. Cubic does not.

If $n = 1$ and $B = 1$, all four models cost at most the budget under the section's convention, so the answer is **cubic**: we ask for the slowest class that still fits.

9 Proof habits start now

Even these easy exercises need correctness arguments. A proof for a simple loop usually has three parts:

1. **Invariant:** what is true before and after every iteration?
2. **Progress:** why does the loop move toward stopping?
3. **Exit:** why does the stopping condition imply the required answer?

For summing an array, the invariant is:

after reading `i` values, `total` is the sum of the first `i` values.

For the threshold process, the invariant is:

after `k` iterations, `x` is the result of applying the operation exactly `k` times.

These are small proofs, but the same style reappears in binary search, greedy algorithms, graph traversal, and dynamic programming.

10 Using the local judge

The course judge is intentionally similar to an online judge:

```
source file
  |
  v
compile if C++
  |
  v
run on fixed tests
  |
  v
run on generated tests
  |
  v
compare stdout with expected output
```

Run the whole section:

```
python3 sections/01_complexity_io_constraints/check.py
```

Run one language for one problem:

```
python3 tools/judge.py \  
  sections/01_complexity_io_constraints/problems/a_sum_constraints \  
  --lang cpp
```

Validate the reference solution:

```
CP_TARGET=solution python3 sections/01_complexity_io_constraints/check.py
```

The reference command is not a shortcut for studying. It exists to check that the tests and judge are coherent.

11 Common mistakes

- Treating $t * n$ as the bound when the statement gives a total sum bound.
- Using `int` in C++ for sums that need `long long`.
- Writing an $O(n^2)$ idea because the sample has $n = 5$.
- Forgetting that sorting is $O(n \log n)$, not linear.
- Making Python parse input line by line when the input is huge.
- Trusting samples as if they were a proof.

12 Before you submit

For every problem in this section, ask:

1. Did I read exactly the input format?
2. Does my loop do work proportional to the real bound?
3. Could the answer overflow 32-bit `int`?
4. Did I print one line per test case?
5. Did I test the smallest case?
6. Did I test a large-value case?

13 Exercises

The local exercises are deliberately small. They train the mechanics that all later sections depend on:

1. `a_sum_constraints`: read many arrays and use the right integer size.
2. `b_until_threshold`: identify a logarithmic simulation.
3. `c_token_budget`: turn complexity estimates into decisions.

Run:

```
python3 sections/01_complexity_io_constraints/check.py
```